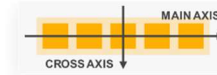


FLEXBOXEN

FLEX CONTAINER



FLEX ITEMS

- gap:** 0 | <length>
To create space between items, without using margin
 - justify-content:** flex-start | flex-end | center | space-between | space-around | space-evenly
To align items along main axis (horizontally, by default)
 - align-items:** stretch | flex-start | flex-end | center | baseline
To align items along cross axis (vertically, by default)
 - flex-direction:** row | row-reverse | column | column-reverse
To define which is the main axis
 - flex-wrap:** nowrap | wrap | wrap-reverse
To allow items to wrap into a new line if they are too large
 - align-content:** stretch | flex-start | flex-end | center | space-between | space-around
Only applies when there are multiple lines (flex-wrap: wrap)
- align-self:** auto | stretch | flex-start | flex-end | center | baseline
To overwrite align-items for individual flex items
 - flex-grow:** 0 | <integer>
To allow an element to grow (0 means no, 1+ means yes)
 - flex-shrink:** 1 | <integer>
To allow an element to shrink (0 means no, 1+ means yes)
 - flex-basis:** auto | <length>
To define an item's width, instead of the width property
 - flex:** 0 1 auto | <int> <int> <len>
Recommended shorthand for flex-grow, -shrink, -basis.
 - order:** 0 | <integer>
Controls order of items. -1 makes item first, 1 makes it last

Fixe Positionierung

- Das Element wird relativ zum Ansichtsbereich (Browserfenster) positioniert.
- Es bleibt auch dann in seiner Position, wenn die Seite gescrollt wird.
- Wie bei der absoluten Positionierung wird es aus dem normalen Dokumentenfluss entfernt.

```
.fixes-element {
  position: fixed;
  top: 20px;
  left: 50px;
}
```

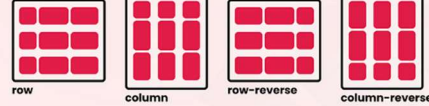
Sticky Positionierung

- Eine Kombination aus relative und fixed
- Das Element ist zuerst relativ positioniert
- Sobald es einen bestimmten Scrollpunkt erreicht, wird es "kleben bleiben" (sticky)
- Scrollt mit, bis es eine definierte Grenze erreicht (z.B. top: 20px)
- Bleibt sichtbar fixiert an dieser Position, bis der übergeordnete Container zu Ende ist

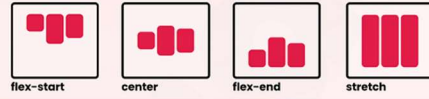
```
.fixes-element {
  position: sticky;
  top: 20px;
}
```

CSS Flexbox

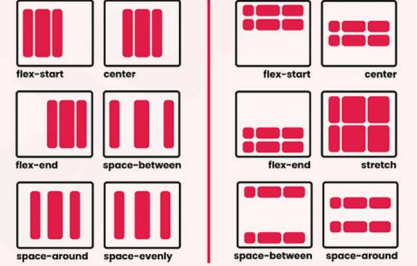
flex-direction



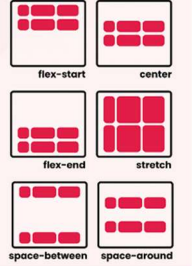
align-items



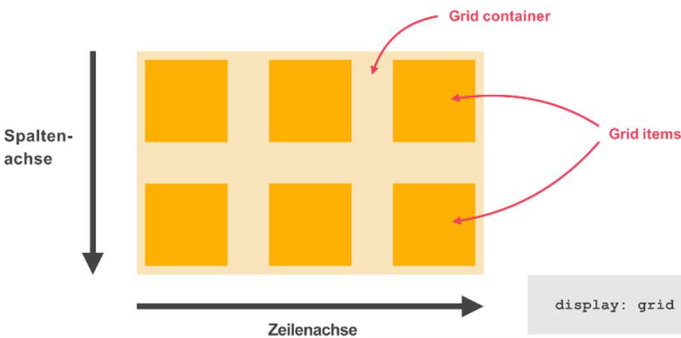
justify-content



align-content



GRID

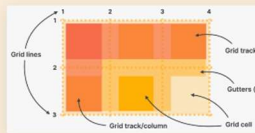


GRID CONTAINER

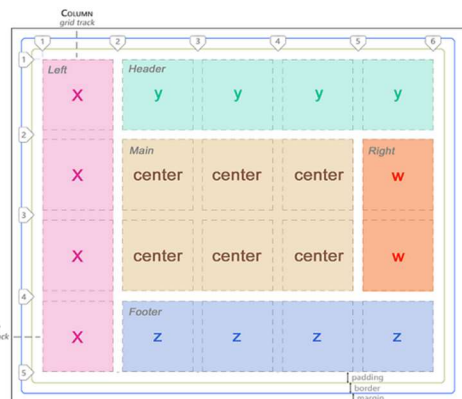
- grid-template-rows:** <track size>*
grid-template-columns: <track size>*
To establish the grid row and column tracks. One length unit for each track. Any unit can be used, new fr fills unused space
- row-gap:** 0 | <length>
column-gap: 0 | <length>
To create empty space between tracks
- justify-items:** stretch | start | center | end
align-items: stretch | start | center | end
To align items inside rows / columns (horizontally / vertically)
- justify-content:** start | start | center | end | ...
align-content: start | start | center | end | ...
To align entire grid inside grid container. Only applies if container is larger than the grid

GRID ITEMS

- grid-column:** <start line> / <end line> | span <number>
grid-row: <start line> / <end line> | span <number>
To place a grid item into a specific cell, based on line numbers. span keyword can be used to span an item across more cells
- justify-self:** stretch | start | center | end
align-self: stretch | start | center | end
To overwrite justify-items / align-items for single items



This list of CSS Grid properties is not exhaustive, but enough to get started.



```
1 <html>
2 <style type = "text/css">
3   #grid { grid-template-columns:
4     'x y y y';
5     'x center center center w';
6     'x center center center w';
7     'x z z z';
8   }
9 </style>
10 </html>
11 <body>
12   <div id = "grid">
13     <div style = "grid-area: x"&>Left</div>
14     <div style = "grid-area: y">Header</div>
15     <div style = "grid-area: z">Footer</div>
16     <div style = "grid-area: w">Right</div>
17     <div style = "grid-area: center">Main</div>
18   </div>
19 </body>
20 </html>
```

Positionen (Mehr auf Seite 2)

Positionierung px vs %

- Prozent %
 - Bezieht sich auf den Prozentsatz der Größe des übergeordneten Elements
 - Gibt die Möglichkeit Größe und Position relativ zu anderen bzw. übergeordneten Elementen anzugeben
 - Eignet sich gut, wenn auf verschiedene Bildschirmgrößen skaliert werden soll
 - Nachteil: kann komplex sein, insbesondere wenn ineinander verschachtelte übergeordnete Elemente vorkommen

Was für Positionierungen gibt es?

Statische Positionierung

- Dies ist die Standardpositionierung für HTML-Elemente.
- Elemente werden entsprechend dem normalen Elementenfluss des Dokuments angeordnet.
- Solche Elemente werden nicht durch die Eigenschaften top, bottom, left oder right beeinflusst.

Relative Positionierung

- Das Element wird relativ zu seiner normalen Position im Dokumentenfluss positioniert.
- Wenn Sie die Eigenschaften top, bottom, left oder right verwenden, wird das Element von seiner normalen Position verschoben.
- Benachbarte oder andere Elemente werden nicht von dem positionierten Element beeinflusst.

```
.relative-element {
  position: relative;
  top: 20px;
  left: 30px;
}
```

Positionierung px vs %

- **Pixel**
 - Absolute Einheit für eine feste Grösse
 - Präzise Kontrolle / Positionierung
 - Unabhängig von anderen Faktoren
 - Eigenen sich gut für Abstände und Grössen
 - Nachteil: bei unterschiedlichen Bildschirmgrössen können Probleme auftreten

Was für Grössen & Abstands-Typen gibt es?

■ Relative Einheiten:

■ Prozent (%)

Bezieht sich auf die Grösse des Elternelements.
Beispiel: width: 50% macht das Element halb so breit wie sein Elterncontainer.

■ em

Schriftgrösse bezieht sich auf die Schriftgrösse des Eltern-Elements.
Bei anderen Eigenschaften (margin, padding, line-height, ...) orientiert sich em an der **eigenen** (berechneten) Schriftgrösse des Elements.

■ rem (Root EM)

Bezieht sich auf die Schriftgrösse des Root-Elements (<html>).
Vorteil: Einheitlich und unabhängig von verschachtelten Elementen.

Was für Grössen & Abstands-Typen gibt es?

■ Absolute Einheiten:

- **Pixel (px):** Kleinste Einheit auf dem Bildschirm.
→ Häufig verwendet für präzise Abstände oder Linien.
- **Punkt (pt):** Ursprünglich aus dem Druckbereich, 1pt = 1/72 Zoll.
→ Am Bildschirm fast wie px, aber weniger gebräuchlich.
- **Zentimeter / Millimeter / Zoll** (cm, mm, inch):
→ Versuchen sich an realen physikalischen Massen zu orientieren, sind aber von Monitor & Software limitiert.
→ Für Web eher ungeeignet.

Pixel (px) im Detail

- Pixel ist eine absolute Masseinheit
- In HTML beträgt die **Standard-Schriftgrösse** meistens **16px**.

Vorteile:

- sehr präzise Kontrolle
- hohe Kompatibilität mit allen Browsern & Geräten

Nachteile:

- Weniger flexibel → mehr Aufwand bei responsivem Design
- schlechte/keine Barrierefreiheit
Schrift lässt sich bei festen Pixelwerten schlecht oder gar nicht anpassen

Root EM (REM) im Detail

- REM ist eine relative Masseinheit
- Sie orientiert sich nicht an der Bildschirmgrösse, sondern an der **Schriftgrösse des Root-Elements** (<html>).
- Standard: **1rem = 16px** (falls der Browser nicht angepasst wurde).

Vorteile:

- Barrierefreiheit
Nutzer können die Standard-Schriftgrösse des Browsers ändern.
- Einheitliches Skalieren: Alle Elemente passen sich im gleichen Verhältnis an.

Nachteile:

- Höhere Komplexität bei der Berechnung von Werten
- Umrechnungsfehler können zu «verdrückten» Werten führen
→ nicht gut, wenn etwas pixelgenau sein muss

Beispiel

```
<body>
  <p>Schriftgrösse vom Body</p>
  <div class="box">
    <p class="mitpx">Schriftgrösse mit PX</p>
    <p class="mitrem">Schriftgrösse mit REM</p>
    <p class="mitem">Schriftgrösse mit EM</p>
  </div>
</body>
```

Schriftgrösse vom Body (10px / überschrieben)

Schriftgrösse Standard Box (10px aus body * 2(em) = 20px)
Schriftgrösse mit PX (20px fix)
Schriftgrösse mit REM (16px aus HTML * 1.25(rem) = 20px)
Schriftgrösse mit EM (10px aus Body * 2(em) aus div * 1.25(em) = 25px)

Annahme: HTML-Standard-Einstellung des Browsers: 16px

```
body {
  font-size: 10px;
}

.box {
  font-size: 2em;
}

.mitpx {
  font-size: 20px;
}

.mitem {
  font-size: 1.25em;
}

.mitrem {
  font-size: 1.25rem;
}
```

EM

- Funktionsweise von EM ähnelt der von REM stark
- EM orientiert sich am darüberliegenden Element, dem Parent
- Beispiel: Hat das Parent font-size: 20px, dann ist 1em = 20px.

Vorteile:

- Sehr flexibel für die Erstellung responsiver Designs
- Ganze Sektionen können auf einmal angepasst werden

Nachteile:

- Komplexität steigt bei verschachtelten Elementen
- Weniger barrierefrei, da es nicht direkt auf die Browser-Standard-einstellungen reagiert

Block – Element – Modifier für komplexe Projekte:

Aufbau

- **Block** → eigenständige, wiederverwendbare Komponente (z. B. `card`, `menu`, `button`)
- **Element** → Bestandteil eines Blocks, das ohne diesen nicht sinnvoll ist.
Schreibweise: `block__element`
Beispiel: `card__title`, `menu__item`
- **Modifier** → Variante oder Zustand eines Blocks oder Elements.
Schreibweise: `block--modifier` oder `block__element--modifier`
Beispiel: `button--active`, `card__title--large`

Absolute Positionierung

- Das Element wird aus dem normalen Dokumentenfluss entfernt und relativ zu seinem enthaltenen Block, unabhängig von seinen „Nachbarn“ positioniert.
- Wenn kein enthaltener Block definiert ist, wird es relativ zum Initial-Containerblock positioniert (normalerweise der Ansichtsbereich).
- Das positionierte Element beeinflusst nicht die Position anderer Elemente.

```
.absolute-element {
  position: absolute;
  top: 20px;
  left: 50px;
}
```

Dies ist ein normaler Absatz.
Dies ist ein relativ positioniertes Element.
Dies ist ein weiterer normaler Absatz.
p.relative-element 258.39 x 18